# Part 1 – Pattern discovery: Member 2 prompt and result

Course 23CSE455 Design Patterns, Team A15
Member: Pranav R Menon (AM.AI.U4AID23017)
Scope: business layer (service/, util/)
Model: Claude Sonnet 5.5 (configured id claude-sonnet-5-5), 2026-10-11
Attached file: `input_source_numbered.txt` (numbered Java source of the baseline application, 20 KB; it is in `Part1/llm/`).

# Prompt

```
[Member 2 prompt - Part 1]
SCOPE: business layer = service/ (AccountService, UserService) and util/ (AccountUtil, UserUtil),
plus the dependency-wiring (@Inject / @ApplicationScoped) used across all classes.
You may name classes outside the scope when they are collaborators, but only report candidates whose
main participants are in this scope.

You are assisting with a university Design Patterns case study (course 23CSE455).

INPUT: the numbered Java source of a Quarkus bank REST API (baseline version), supplied as
input_source_numbered.txt. Every source line starts with its real line number followed by '|'.

TASK (Part 1, pattern discovery) - restricted to the scope given below:
1. Find every pattern candidate in your scope: GoF patterns, GRASP patterns, enterprise/architecture
   patterns, and framework idioms. Give the exact recognised pattern name and its category.
2. For each candidate give a meaningful instance name (one collaboration, e.g. "Book Return Notification",
   not just "Strategy").
3. Map every pattern role to the ACTUAL class / interface / method in the code. Do not invent classes.
   If a role is combined or absent, say "not separately declared".
4. Give a file path and line range for every participant, using this exact row format:
       ROLE | SYMBOL | path:start-end
   where path is relative to src/main/java/br/com/michaelcruz/ (or src/main/resources/).
5. Give concise evidence from the code (relationships, calls, delegation, behaviour).
   A class name or an annotation alone is NOT sufficient evidence.
6. Give a verdict for each candidate: CONFIRMED, REJECTED or UNCERTAIN, with a reason.
   Two concrete classes in one collaboration are ONE instance.
7. Finish with the counts for your scope: confirmed pattern types, confirmed instances,
   concrete participant classes, rejected candidates, uncertain candidates.
Do not use any knowledge about this repository other than the supplied code.
```

# Result

```
# Member 2 - raw model output (scope: service/, util/, dependency wiring)

## Candidate 1 - "Banking Use-Case Service Layer" - Service Layer - CONFIRMED
ServiceLayer | AccountService | service/AccountService.java:18-85
ServiceLayer | UserService | service/UserService.java:15-46
Client | AccountController | rest/AccountController.java:13-50
Client | UserController | rest/UserController.java:10-21
Collaborator | AccountDAO (field accountDAO) | service/AccountService.java:20-21
Collaborator | UserService (field userService) | service/AccountService.java:23-24
Collaborator | AccountUtil (field accountUtil) | service/AccountService.java:26-27
Collaborator | UserDAO (field userDAO) | service/UserService.java:18-19
Collaborator | UserUtil (field userUtil) | service/UserService.java:21-22

Evidence: the services expose the use cases createAccount (30-34), depositToAccount (36-58), withdraw (60-79),
listAllAccounts (81-83), createUser (UserService 25-34) and getUserDetails (36-44). AccountService.createAccount
coordinates three collaborators: userService.getUserDetails (line 31), accountUtil.convertToAccount (line 32) and
accountDAO.save (line 33). Controllers call only these services.

## Candidate 2 - "Account Deposit and Withdrawal Script" - Transaction Script - CONFIRMED
TransactionScript | AccountService.depositToAccount | service/AccountService.java:36-58
TransactionScript | AccountService.withdraw | service/AccountService.java:60-79
TransactionScript | AccountService.createAccount | service/AccountService.java:30-34
TransactionScript | UserService.createUser | service/UserService.java:25-34
RowData | Account (anemic entity, accessors only) | persistence/model/Account.java:5-44

Evidence: each use case is one procedure. The business rules live inside the service methods: amount > 0
(line 43), the savings bonus `newBalance += 0.5` for CONTA_POUPANCA (lines 46-48), and the withdrawal rule
`amount > 0 && balance > amount` (line 67). Account (lines 5-44) holds data and getters/setters only and has no
behaviour, so the logic cannot be on the domain object.
```

## Candidate 3 - "DTO <-> Entity Assembly" - Assembler role of the Data Transfer Object pattern - CONFIRMED
(same collaboration as Member 1's DTO candidate; the assembler half)
Assembler | UserUtil.convertToUserDTO | util/UserUtil.java:18-28
Assembler | AccountUtil.convertToAccount | util/AccountUtil.java:21-33
Assembler | UserService.createUser (inline DTO to User mapping) | service/UserService.java:25-34
Evidence: convertToUserDTO copies the five fields of User into a UserDTO (lines 21-25); convertToAccount copies type and balance from AccountDTO and attaches a User that carries only the id from the UserDTO (lines 24-30); UserService.createUser copies the DTO fields into a new User by hand (lines 28-31).

## Candidate 4 - "CDI Field Injection Wiring" - Dependency Injection (CDI / IoC framework idiom) - CONFIRMED
Container | not separately declared (Quarkus ArC CDI container, dependency)
InjectionPoint | AccountController.accountService | rest/AccountController.java:16-17
InjectionPoint | UserController.userService | rest/UserController.java:13-14
InjectionPoint | AccountService.accountDAO | service/AccountService.java:20-21
InjectionPoint | AccountService.userService | service/AccountService.java:23-24
InjectionPoint | AccountService.accountUtil | service/AccountService.java:26-27
InjectionPoint | UserService.userDAO | service/UserService.java:18-19
InjectionPoint | UserService.userUtil | service/UserService.java:21-22
InjectionPoint | AccountDAO.entityManager | persistence/dao/AccountDAO.java:16-17
InjectionPoint | UserDAO.entityManager | persistence/dao/UserDAO.java:16-17
InjectionPoint | AccountUtil.accountDAO | util/AccountUtil.java:17-18
InjectionPoint | UserUtil.userDAO | util/UserUtil.java:14-15
ManagedBean | AccountService | service/AccountService.java:18-85
ManagedBean | UserService | service/UserService.java:15-46
ManagedBean | AccountUtil | util/AccountUtil.java:14-35
ManagedBean | UserUtil | util/UserUtil.java:12-30
ManagedBean | AccountDAO | persistence/dao/AccountDAO.java:13-45
ManagedBean | UserDAO | persistence/dao/UserDAO.java:13-38

Evidence: no class creates a collaborator with ‘new‘ (the only ‘new‘ expressions are for Account, User and DTO data objects); every collaborator is a field marked @Inject. AccountService can therefore be given any AccountDAO.
Note: AccountUtil.accountDAO and UserUtil.userDAO are injected but never used (dead injection points).

## Rejected / uncertain
- Strategy for the savings bonus (AccountService lines 46-48) - REJECTED: a single ‘if‘ on an enum value, no strategy interface and no family of interchangeable algorithms.
- Factory Method / Simple Factory for AccountUtil.convertToAccount (21-33) - REJECTED: it maps data fields; there is no product hierarchy and the caller does not choose among products.
- Adapter for the *Util classes - REJECTED: they convert data between two unrelated types, they do not make an incompatible interface usable behind a target interface.
- GoF Singleton for the @ApplicationScoped beans - REJECTED: no private constructor or static instance; lifetime is decided by the container (already covered by Dependency Injection).
- Facade for AccountService.createAccount (30-34) - UNCERTAIN: it hides three collaborators behind one call but that is the normal job of a service method (already counted as Service Layer).

## Counts for my scope
confirmed pattern types: 4 (Service Layer, Transaction Script, DTO assembler [same collaboration as Member 1's DTO], Dependency Injection); confirmed instances: 4 before merging with Member 1 (3 after merging the DTO halves); concrete participant classes: 2 services + 2 script hosts (same two classes) + 2 assemblers + 8 injecting classes; rejected: 4; uncertain: 1.